AI Thinker VC-02 Voice Recognition Integration Plan

Akshi Robot — Keyboard → Voice Input Migration

Document Version: 1.0
Date: 2026-05-04
Author: Dinujaya S.
Project: Akshi Educational Robot
Hardware Target: Raspberry Pi + AI Thinker VC-02 Module

---

1. Executive Summary

This document details the complete migration of the Akshi Robot's input system from physical keyboard commands to the **AI Thinker VC-02** offline voice recognition module. The VC-02 communicates with the Raspberry Pi over **UART serial** (`/dev/serial0`) at **9600 baud**, sending 2-byte hex frames (`0xAA` + command byte) when a trained voice command is recognized.

The integration is designed as a **zero-change architecture** for all 14 UI screen files. By injecting the serial listener directly into the existing `KeyboardManager` singleton, every screen automatically gains voice control without modifying a single line of screen code.

---

2. Hardware Architecture

2.1 Wiring Diagram (Raspberry Pi → VC-02)

> [!CAUTION] The VC-02 operates at **3.3V logic**. Do NOT connect to 5V — it will permanently damage the module.

2.2 Raspberry Pi UART Setup

Before using `/dev/serial0`, the Raspberry Pi's serial console must be disabled:

```
# Step 1: Open raspi-config sudo raspi-config # Step 2: Navigate to: # Interface Options → Serial Port # - "Login shell over serial?" → NO # - "Serial port hardware enabled?" → YES # Step 3: Reboot sudo reboot # Step 4: Verify the serial port exists ls -la /dev/serial0 # Should show: /dev/serial0 -> ttyS0 (or ttyAMA0 on Pi 4/5)
```

2.3 Install pyserial

```
pip install pyserial
```

---

3. Voice Command Mapping

3.1 Current Keyboard Inputs (Audit)

The full codebase audit identified **12 distinct logical actions** used across all screens:

#	Keyboard Key	Internal Action	Screen(s) Used In	Purpose
1	`W`	`WAKE`	idle_screen, session_complete	Wake robot / restart session
2	`1`	`1`	evaluate_screen, evaluation	Select MCQ Option 1
3	`2`	`2`	evaluate_screen, evaluation	Select MCQ Option 2
4	`3`	`3`	evaluate_screen, evaluation	Select MCQ Option 3
5	`4`	`4`	evaluate_screen, evaluation	Select MCQ Option 4

6	`Enter`	`ENTER`	explore, explain, elaborate, engage, student_call, break_screen	Proceed / Confirm
7	`C`	`CONTINUE`	teacher_intervention, student_call, session_complete	Teacher continues
8	`S`	`SKIP`	calibration_screen, evaluate_screen (L3)	Skip current task
9	`B`	`BREAK`	evaluate_screen (L2+)	Request a break
10	`P`	`P`	kinestatic	Teacher override: PASS
11	`F`	`F`	kinestatic	Teacher override: FAIL
12	`A-Z`	`A`, `B`, ...	teacher_select	Select teacher by letter

3.2 VC-02 Hex Command Assignment

The VC-02 sends a **2-byte frame**: `[0xAA] [CMD]`. We assign each voice trigger a unique command byte:

Hex Code	Voice Trigger Phrase	Maps To Action	Used By
`AA 01`	*"Hey Akshi"* or *"Wake up"*	`WAKE`	Idle / Session Complete
`AA 02`	*"Option one"*	`1`	Evaluate MCQ
`AA 03`	*"Option two"*	`2`	Evaluate MCQ

`AA 04`	*"Option three"*	`3`	Evaluate MCQ
`AA 05`	*"Option four"*	`4`	Evaluate MCQ
`AA 06`	*"Next"* or *"Continue"*	`ENTER`	All proceed screens
`AA 07`	*"Teacher continue"*	`CONTINUE`	Teacher Intervention
`AA 08`	*"Skip"*	`SKIP`	Calibration / Evaluate L3
`AA 09`	*"Take a break"*	`BREAK`	Evaluate L2+
`AA 0A`	*"Pass"*	`P`	Kinesthetic (Teacher)
`AA 0B`	*"Fail"*	`F`	Kinesthetic (Teacher)
`AA 0C`	*"Select teacher"*	`CONTINUE`	Teacher Select (fallback)

> [!NOTE] Teacher selection (`A-Z` keys) is special. Since only 1-3 teachers exist in practice, use `AA 0C` as a generic "confirm first teacher" shortcut, or hardcode specific teacher names as additional commands if needed.

---

4. VC-02 Firmware Programming

4.1 Programming Tool

Use the **AI Thinker VC-02 SDK** (Windows application) to flash custom voice commands. Connect the VC-02 to a PC via USB-to-UART adapter.

4.2 Configuration Steps

1. Open the VC-02 SDK tool on Windows.
2. Select the correct COM port for the USB-to-UART adapter.
3. For each command entry, configure:
   - **Trigger Phrase**: The spoken phrase (e.g., "Option one")
   - **Response Frame**: The hex bytes to transmit (e.g., `AA 02`)
   - **Confidence Threshold**: Set to **50-60%** for noisy classroom environments

4. Program all 12 commands from the table above.
5. Flash the firmware to the VC-02 module.
6. Test each command by speaking the trigger phrase and verifying the hex output on a serial monitor.

4.3 Example SDK Configuration Table

```
Index | Trigger Word | TX Data (Hex) | Confidence
------+--------------+---------------+----------- 01 | hey akshi | AA 01 | 55% 02 | option one | AA 02 | 55% 03 | option two | AA 03 | 55% 04 | option three | AA 04 | 55% 05 | option four | AA 05 | 55% 06 | next | AA 06 | 50% 07 | teacher continue | AA 07 | 55% 08 | skip | AA 08 | 60% 09 | take a break | AA 09 | 55% 10 | pass | AA 0A | 60% 11 | fail | AA 0B | 60% 12 | select teacher | AA 0C | 55%
```

> [!IMPORTANT] Set confidence higher (60%) for destructive commands like "Skip", "Pass", and "Fail" to prevent accidental triggers.

---

5. Software Implementation

5.1 Architecture Overview

```
graph TD A["VC-02 Module<br/>(UART TX)"] -->|"Serial bytes<br/>AA 06"] B["SerialInputThread<br/>(Background Thread)"] B -->|"Qt Signal<br/>voice_command_received"] C["KeyboardManager<br/>(Main UI Thread)"] D["Physical Keyboard<br/>(Qt KeyPress Event)"] -->|"handle_key_press()"] C C -->|"active_handler(action)"] E["Active Screen<br/>(e.g. evaluate_screen)"] style A fill:#4CAF50,color:#fff style B fill:#FF9800,color:#fff style C fill:#1976D2,color:#fff style D fill:#9E9E9E,color:#fff style E fill:#7B1FA2,color:#fff
```

The key insight: **both keyboard and voice inputs funnel into the same `KeyboardManager.active_handler()`**. No screen code changes are needed.

> [!WARNING] PySide6 crashes if you update UI widgets from a background thread. The `SerialInputThread` MUST use a Qt `Signal` to safely bridge data from the serial thread to the main UI thread.

5.2 Code Change #1: `core/keyboard_manager.py` (FULL REPLACEMENT)

This is the **only core file** that needs significant changes. The replacement adds a background serial listener thread that emits a Qt Signal when a voice command arrives.

```
import sys import threading from PySide6.QtCore import Qt, QObject, Signal #
████████████████████████████████████████████████████████████ #
Toggle: Set to True on the Raspberry Pi, False on dev laptop USE_VOICE_INPUT =
False #
████████████████████████████████████████████████████████████ class
VoiceSignalBridge(QObject): """Thread-safe bridge: emits a Qt Signal from the
serial thread so the main UI thread can safely process voice commands."""
voice_command_received = Signal(str) class KeyboardManager: def __init__(self):
self.active_handler = None # ██ Keyboard Mappings (unchanged) ██
self.key_mapping = { Qt.Key_W: "WAKE", Qt.Key_S: "SKIP", Qt.Key_B: "BREAK",
Qt.Key_C: "CONTINUE", Qt.Key_Return: "ENTER", Qt.Key_Enter: "ENTER", } for i in
range(26): key_code = Qt.Key_A + i if key_code not in self.key_mapping:
self.key_mapping[key_code] = chr(ord('A') + i) for i in range(10):
self.key_mapping[Qt.Key_0 + i] = str(i) # ██ Voice Command Hex Mappings ██
self.voice_hex_mapping = { 0x01: "WAKE", 0x02: "1", 0x03: "2", 0x04: "3", 0x05:
"4", 0x06: "ENTER", 0x07: "CONTINUE", 0x08: "SKIP", 0x09: "BREAK", 0x0A: "P",
0x0B: "F", 0x0C: "CONTINUE", } # ██ Voice Input Thread (Raspberry Pi only) ██
self.voice_bridge = VoiceSignalBridge()
self.voice_bridge.voice_command_received.connect( self._on_voice_command ) if
USE_VOICE_INPUT: self._start_serial_listener() def _start_serial_listener(self):
"""Start background thread to read VC-02 serial data.""" def listen(): import
serial try: ser = serial.Serial( '/dev/serial0', baudrate=9600, timeout=1 )
print("[Voice Input] Serial port opened. Listening...") except Exception as e:
print(f"[Voice Input] FAILED to open serial: {e}") return while True: try: if
ser.in_waiting >= 2: start_byte = ser.read(1)[0] cmd_byte = ser.read(1)[0] if
start_byte == 0xAA: action = self.voice_hex_mapping.get(cmd_byte) if action:
print(f"[Voice Input] Received: " f"AA {cmd_byte:02X} → {action}") # Emit
signal to main thread self.voice_bridge \ .voice_command_received.emit(action)
else: print(f"[Voice Input] Unknown cmd: " f"AA {cmd_byte:02X}") else: import
time time.sleep(0.03) except Exception as e: print(f"[Voice Input] Read error:
{e}") import time time.sleep(1) thread = threading.Thread(target=listen,
daemon=True) thread.start() print("[Voice Input] Serial listener thread
started.") def _on_voice_command(self, action: str): """Called on the MAIN
thread when a voice command is received.""" print(f"[Voice Input] Dispatching
action: {action}") if self.active_handler: self.active_handler(action) def
register_handler(self, handler_function): """Register the current active
screen's key handler.""" self.active_handler = handler_function def
unregister_handler(self): """Unregister the active screen's handler."""
self.active_handler = None def handle_key_press(self, event): """Process
keyboard key press (unchanged behavior).""" key = event.key() mapped_action =
self.key_mapping.get(key) if not mapped_action and event.text().upper():
mapped_action = event.text().upper() if self.active_handler and mapped_action:
self.active_handler(mapped_action) # Global singleton instance keyboard_manager
= KeyboardManager()
```

5.3 Code Change #2: `main.py` — No Changes Required!

The `GlobalKeyListener` event filter in `main.py` calls `keyboard_manager.handle_key_press(event)` for keyboard input. The voice input runs

independently via the serial thread and the Qt Signal. **No changes are needed in main.py.**

5.4 Code Change #3: Screen Files — No Changes Required!

Every screen (idle, evaluate, kinestatic, etc.) uses this exact pattern:

```
keyboard_manager.register_handler(handle_key_press)
```

Since both keyboard and voice inputs route through the same `active_handler`, **zero screen files need modification**.

---

6. Deployment Toggle

The system supports **dual-input mode** via a single boolean flag:

```
# In core/keyboard_manager.py, line 9: USE_VOICE_INPUT = False # Development laptop (keyboard only) USE_VOICE_INPUT = True # Raspberry Pi (voice + keyboard)
```

When `USE_VOICE_INPUT = True`: - Voice commands from the VC-02 are active - Keyboard input **still works** as a fallback (useful for debugging)

When `USE_VOICE_INPUT = False`: - Serial port is never opened (no `pyserial` import error on laptops) - Keyboard-only mode for development

---

7. Testing Procedure

7.1 Phase 1: Serial Verification (No UI)

Create a test script to verify wiring before integrating:

```
# test_serial.py — Run on Raspberry Pi import serial import time ser = serial.Serial('/dev/serial0', baudrate=9600, timeout=1) print("Listening for VC-02 commands... Speak now!") EXPECTED = { 0x01: "WAKE", 0x02: "1", 0x03: "2", 0x04: "3", 0x05: "4", 0x06: "ENTER", 0x07: "CONTINUE", 0x08: "SKIP", 0x09: "BREAK", 0x0A: "P", 0x0B: "F", } while True: if ser.in_waiting >= 2: start = ser.read(1)[0] cmd = ser.read(1)[0] print(f"Raw: {start:02X} {cmd:02X}", end=" → ") if start == 0xAA and cmd in EXPECTED: print(f"■ {EXPECTED[cmd]}") else: print("■ Unknown or malformed") time.sleep(0.03)
```

Success criteria: All 11 voice commands produce the correct ■ output.

7.2 Phase 2: Integration Test

1. Set `USE_VOICE_INPUT = True` in `keyboard_manager.py`
2. Set `USE_DUMMY = True` in `backend_manager.py`
3. Run `python main.py`
4. Test each voice command on its corresponding screen:

Test	Screen	Say	Expected Result
T1	Idle	"Hey Akshi"	Transitions to Greeting
T2	Student Call	"Next"	Transitions to Calibration
T3	Calibration	"Skip"	Skips to task flow
T4	Evaluate L1	"Option one"	Selects answer 1
T5	Evaluate L1	"Option two"	Selects answer 2
T6	Evaluate L2	"Take a break"	Goes to Break screen
T7	Break (return)	"Next"	Returns to evaluation
T8	Elaborate	"Next"	Proceeds to re-evaluate
T9	Kinesthetic	"Pass"	Teacher passes student
T10	Kinesthetic	"Fail"	Teacher fails student
T11	Teacher Intervention	"Teacher continue"	Proceeds to next student
T12	Session Complete	"Hey Akshi"	Returns to Idle

---

8. File Change Summary

File	Change Type	Description
`core/keyboard_manager.py`	**MODIFIED**	Add `VoiceSignalBridge`, serial thread, hex mapping
`aiThinker.py`	**DELETED**	Logic absorbed into `keyboard_manager.py`
`test_serial.py`	**NEW**	Standalone serial verification script
All screen files	**UNCHANGED**	Zero modifications needed
`main.py`	**UNCHANGED**	Zero modifications needed

---

9. Risk Mitigation

Risk	Mitigation
VC-02 misrecognizes command in noisy classroom	Set confidence threshold to 55-60%; use `input_enabled` guards in screens
Serial port not available on dev laptop	`USE_VOICE_INPUT = False` prevents import errors
PySide6 thread crash	Qt `Signal` bridge ensures all UI updates happen on the main thread
VC-02 module disconnects mid-session	Serial read loop has `try/except` with 1-second retry
Teacher needs keyboard fallback	Keyboard input is always active alongside voice input

---

10. Summary

The entire migration requires changing **exactly 1 file** (`core/keyboard_manager.py`) and programming **12 voice commands** on the VC-02 module. The architecture ensures:

1. **Backward compatibility** — keyboard still works for development
2. **Thread safety** — Qt Signal bridge prevents UI crashes
3. **Zero screen changes** — all 14 screens work automatically
4. **Graceful fallback** — if the serial port fails, the app continues with keyboard-only input